Σενάρια φλοιού (Shell scripts) Β

Τι περιέχουν τα προγράμματα φλοιού

- Μεταβλητές
- Λογικές δομές (if, case …)
- Δομές επανάληψης (while, for, until)
- Συναρτήσεις (functions)
- Σχόλια
- Εκτέλεση προγραμμάτων/εντολών

Μεταβλητές

- Στο shell υπάρχουν μεταβλητές.
- Μπορούμε να ορίσουμε οποιαδήποτε μεταβλητή.
- Οι προκαθορισμένες μεταβλητές ονομάζονται μεταβλητές περιβάλλοντος.
- Όλες οι μεταβλητές είναι τύπου "string" και δεν καθορίζουμε τον τύπο τους.

Μεταβλητές

- Ως ονόματα μεταβλητών ο χρήστης μπορεί να θέσει οποιονδήποτε συνδυασμό γραμμάτων και αριθμών (αρχίζοντας από γράμμα). Δεν επιτρέπονται κενά και άλλοι ειδικοί χαρακτήρες που μπορεί να έχουν κάποια ειδική σημασία για το κέλυφος (π.χ. $ \ # ; κ.ο.κ.).

Environment variables

$ env

PWD=/home/frag
SHELL=/bin/tcsh
TERM=xterm
USER=frag
PATH=/usr/sbin:usr/bsd:/usr/bin:/usr/X11:/usr/freeware
/bin:/usr/local/bin:/usr/java/bin:
MAIL=/var/mail/frag
LOGNAME=frag
EDITOR=/bin/vi
SSH2_CLIENT=195.251.123.....

- Όταν ένας χρήστης κάνει login στο σύστημα τότε το σύστημα καθορίζει ορισμένες από τις μεταβλητές περιβάλλοντος που δείχνουν το home directory, το shell που εκτελείται κλπ
- Ορισμένες από αυτές τις μεταβλητές υπάρχουν ως πεδία στη γραμμή που αντιστοιχεί στον χρήστη στο password file π.χ.

```
bash-2.05a $ grep $USER /etc/passwd
frag:x:2224:2224:Fragkiskos
   Papadopoulos:/home/frag:/usr/local/bin/bash

bash-2.05a$ echo $USER
frag
bash-2.05a$ echo $HOME
/home/frag
```

Αρχεία εκκίνησης στο bash (startup files)

- Αρχικά διαβάζεται και εκτελείται το αρχείο **/etc/profile** , εφόσον αυτό υπάρχει. Το αρχείο αυτό είναι το εξ ορισμού αρχείο διαμόρφωσης φλοιού για οποιονδήποτε χρήστη του συστήματος.
- Το home directory κάθε χρήστη μπορεί περιέχει ένα ή περισσότερα από τα ακόλουθα bash startup files, τα οποία περιέχουν εντολές που εφαρμόζονται μόνον για την τρέχουσα χρήση του συστήματος:
  - ~/.bash_profile,
  - ~/.bash_login,
  - ~/.profile,
  - ~/bash_rc, και
  - ~/.bash_logout

- Login shell : δημιουργείται κάθε φορά που γίνεται login σε ένα account.

- Όταν ξεκινά ένα login shell, το bash συμβουλεύεται κατά σειρά ορισμένα από τα παρακάτω αρχεία :
  1. /etc/profile
  2. ~/.bash.profile
  3. ~/.bash_login
  4. ~/.profile

- **Η προσθήκη μιας εντολής που θα εκτελείται αυτόματα όταν κάθε χρήστης του συστήματος κάνει login πρέπει να γίνεται στο αρχείο /etc/profile. Ωστόσο χρειάζονται δικαιώματα superuser για την επεξεργασία αυτού του αρχείου.**
- Η προσθήκη μιας εντολής που θα εκτελείται όταν ξεκινά ένας login φλοιός για το δικό μας account, πρέπει να γίνει στο αρχείο ~/.bash.profile.
- Οι εντολές που θα εκτελούνται μόνον μετά την εκτέλεση των εντολών που υπάρχουν στα ~/.bash.profile και ~/.bash_login θα πρέπει να περιλαμβάνονται στο αρχείο ~/.profile.

Τα εισαγωγικά

- Γενικώς τα εισαγωγικά αγνοούνται από το shell, και δεν περιλαμβάνονται στις ίδιες τις τιμές των μεταβλητών που δίνει το shell

```
bash-2.05a$ echo "TEST"
TEST
bash-2.05a$ echo 'TEST'
TEST
bash-2.05a$ echo "TEST     A"
TEST     A
bash-2.05a$ echo TEST     A
TEST A
(Το shell αγνοεί τα πολλαπλά κενά αν αυτά δεν είναι
  μέσα σε εισαγωγικά)
```

Τα εισαγωγικά

- Καλό είναι όταν αναθέτουμε τιμή σε μια μεταβλητή ή όταν χρησιμοποιούμε μια μεταβλητή να χρησιμοποιούμε εισαγωγικά

```
bash-2.05a$ a='test    1'
bash-2.05a$ echo $a
test 1
bash-2.05a$ echo "$a"
test    1
bash-2.05a$
```

Τα εισαγωγικά

- Τα απλά εισαγωγικά ('single quotes') άρουν την μεταχαρακτηρική ιδιότητα όλων των συμβόλων εκτός από τον εαυτό τους.
- Τα διπλά εισαγωγικά ("double quotes") άρουν την μεταχαρακτηρική ιδιότητα όλων των συμβόλων εκτός από τον εαυτό τους, την ανάποδη κάθετο (backslash \ ) και το δολάριο ($).
- Τα ανάποδα εισαγωγικά (` back quotes`) προκαλούν την εκτέλεση της εντολής που περικλείουν.

Τα εισαγωγικά

```
bash-2.05a$ a='test    1'
bash-2.05a$ echo $a
test 1
bash-2.05a$ echo "$a"
test    1
bash-2.05a$ echo '$a'
$a
bash-2.05a$ echo "\$a"
$a
```

Χρήση απλών & διπλών εισαγωγικών

```
$echo "My host name is $HOSTNAME"
   .My host name is apollo.
$echo 'My host name is $HOSTNAME'
   My host name is $HOSTNAME
```

Τα εισαγωγικά

- Το shell όταν βρίσκει το χαρακτήρα $, θεωρεί ότι αυτό που ακολουθεί (μέχρι κάποιον ειδικό χαρακτήρα ή κενό) είναι όνομα μεταβλητής, και το αντικαθιστά με την τιμή του.

```
bash-2.05a$ a='test    1'
bash-2.05a$ echo $abb

bash-2.05a$ echo "$a"bb
test    1bb
bash-2.05a$ echo ${a}bb
test 1bb
bash-2.05a$ echo $a"bb"
test 1bb
bash-2.05a$ echo "${a}bb"
test    1bb
```

Συχνά χρησιμοποιούνται τα {} για να περικλείουν ονόματα μεταβλητών ώστε να ξεχωρίζουν από το string που ακολουθεί

Τα εισαγωγικά

- Τα ανάποδα εισαγωγικά εκτελούν το string που περιέχουν (σαν εντολή), δεν εμφανίζεται τίποτα στην κανονική έξοδο, και «επιστρέφουν» ότι έχει στείλει η εντολή στην έξοδό της.

```
bash-2.05a$ date
Mon Nov 27 18:39:28 EET 2006
bash-2.05a$ a=`date`
bash-2.05a$ echo "$a"
Mon Nov 27 18:39:35 EET 2006
bash-2.05a$ a=`ls|wc`
bash-2.05a$ echo "$a"
             22              22             214
```

Τα εισαγωγικά

```
bash-2.05a$ b=`ls -l`
bash-2.05a$ echo $b
total 32 -rw-r--r-- 1 frag it 314 Jan 11 2003 cc -rwxr--r-- 1
   frag it 77 Nov 20 19:17 file1 -rwxr-xr-x 1 frag it 61 Nov
   20 19:34 file2 -rw-r--r-- 1 frag it 183 Jan 8 2003 list
bash-2.05a$ echo "$b"
total 32
-rw-r--r--    1 frag it          314 Jan 11  2003 cc
-rwxr--r--    1 frag it           77 Nov 20 19:17 file1
-rwxr-xr-x    1 frag it           61 Nov 20 19:34 file2
-rw-r--r--    1 frag it          183 Jan  8  2003 list
```

Τα εισαγωγικά

Στο bash συνώνυμο των ` ` (ανάποδων εισαγωγικών)είναι το $().

```
bash-2.05a$ b=$(ls -l)
bash-2.05a$ echo $b
total 32 -rw-r--r-- 1 frag it 314 Jan 11 2003 cc -rwxr--r-- 1
   frag it 77 Nov 20 19:17 file1 -rwxr-xr-x 1 frag it 61 Nov
   20 19:34 file2 -rw-r--r-- 1 frag it 183 Jan 8 2003 list
bash-2.05a$ echo "$b"
total 32
-rw-r--r--    1 frag it          314 Jan 11  2003 cc
-rwxr--r--    1 frag it           77 Nov 20 19:17 file1
-rwxr-xr-x    1 frag it           61 Nov 20 19:34 file2
-rw-r--r--    1 frag it          183 Jan  8  2003 list
```

Τα εισαγωγικά

- Για το shell όλες οι μεταβλητές είναι τύπου string.
- Το shell δεν ξέρει να κάνει μαθηματικές πράξεις.
- Πράξεις με ακεραίους μπορούμε να κάνουμε με την εντολή expr

```
bash-2.05a$ expr 1 + 2
3
bash-2.05a$ a=1
bash-2.05a$ b=2
bash-2.05a$ expr $a + $b
3
bash-2.05a$ c=`expr $a + $b`
bash-2.05a$ echo $c
3
bash-2.05a$
```

- Το bash μπορεί να κάνει πράξεις με ακεραίους χρησιμοποιώντας τα $(( )):

```
bash-2.05a$ c=$(($a + $b))
bash-2.05a$ echo $c
3
```

Τα εισαγωγικά

- Προσοχή όταν μια μεταβλητή δεν έχει οριστεί:

```
bash-2.05a$ a=1
bash-2.05a$ expr $a + $w
Syntax error
bash-2.05a$ expr 1 +
Syntax error
bash-2.05a$ expr "$a" + "$w"
non-numeric argument
bash-2.05a$ k=`expr $c + $w`
Syntax error
```

Προγραμματισμός κελύφους

- **ΜΕΤΑΒΛΗΤΕΣ**
  - ***Μεταβλητές περιβάλλοντος :*** είναι τμήμα του περιβάλλοντος του συστήματος και δεν χρειάζεται να ορισθούν. Μπορούν να χρησιμοποιηθούν από ένα πρόγραμμα φλοιού ενώ ορισμένες (π.χ. PATH) μπορούν επίσης να τροποποιηθούν.
  - ***Μεταβλητές φλοιού :*** ορίζονται, τροποποιούνται και χρησιμοποιούνται σε ένα σενάριο φλοιού.

Μεταβλητές περιβάλλοντος

- Αρχικοποιούνται κατά την εκκίνηση (login)
- Μεταβιβάζονται σε όλες τις διεργασίες που εκτελούνται από τον τρέχοντα φλοιό (current shell)
- Χρησιμοποιούνται για την αποθήκευση πληροφοριών που χρειάζεται να γνωρίζουν άλλα προγράμματα
- Συνήθως τους αποδίδονται αναγνωριστικά με κεφαλαία γράμματα (upper case identifiers)

Μεταβλητές φλοιού

- Αρχικοποιούνται από τον φλοιό,
- Δεν μεταβιβάζονται στις διεργασίες που εκτελούνται από τον τρέχοντα φλοιό,
- Συνήθως τους αποδίδονται αναγνωριστικά με μικρά γράμματα (lower case identifiers)

Μεταβλητές

- Μπορούμε να μετατρέψουμε μια μεταβλητή φλοιού σε μεταβλητή περιβάλλοντος με την εντολή export:

```
bash-2.05a$ A=TEST
bash-2.05a$ echo $A
TEST
bash-2.05a$ bash
bash-2.05a$ echo $A

bash-2.05a$ exit
exit
bash-2.05a$ echo $A
TEST
bash-2.05a$ export A
bash-2.05a$ bash
bash-2.05a$ echo $A
TEST
bash-2.05a$ exit
exit
```

Χρήση μεταβλητών σε προγράμματα κελύφους

- Για να χρησιμοποιηθεί μια μεταβλητή πρέπει να γνωρίζουμε : πώς να της αποδώσουμε μια τιμή και πώς να *διαβάσουμε* την τιμή μιας μεταβλητής.
- Υπάρχουν 2 τρόποι για να δώσουμε τιμή σε μια μεταβλητή :
  1. χρησιμοποιώντας απευθείας εκχώρηση
  2. χρησιμοποιώντας την εντολή read
- Χρησιμοποιούμε την τιμή μιας μεταβλητής με τον χαρακτήρα $ πριν το όνομα της μεταβλητής.

1. Απευθείας εκχώρηση τιμής (direct assignment)

```
myemail='studin@cut.ac.cy'
```

- Όταν κάνουμε εκχώρηση δεν πρέπει να υπάρχει "space" πριν ή μετά το "="

```
bash-2.05a$ A = 5
bash: A: command not found
bash-2.05a$ A= 5
bash: 5: command not found
bash-2.05a$ A=5
```

2. Χρησιμοποιώντας την εντολή read

- Η εντολή read εκχωρεί σε μια μεταβλητή οτιδήποτε εισάγεται από το πληκτρολόγιο, ακολουθούμενο από μια αλλαγή γραμμής.

```
bash-2.05a$ read b
foo         bar
bash-2.05a$ echo $b
foo bar
bash-2.05a$ echo "$b"
foo         bar
bash-2.05a$
```

Χρησιμοποιώντας την εντολή read

read_demo.bash

```
#!/bin/bash

echo -n "Enter some text > "
read text
echo "You entered: $text"
```

Η επιλογή -n στην echo δεν επιτρέπει την αλλαγή γραμμής.

```
$ ./read_demo.bash
Enter some text > this is some text
You entered: this is some text
```

Αριθμητικές πράξεις (bash)

```
#!/bin/bash

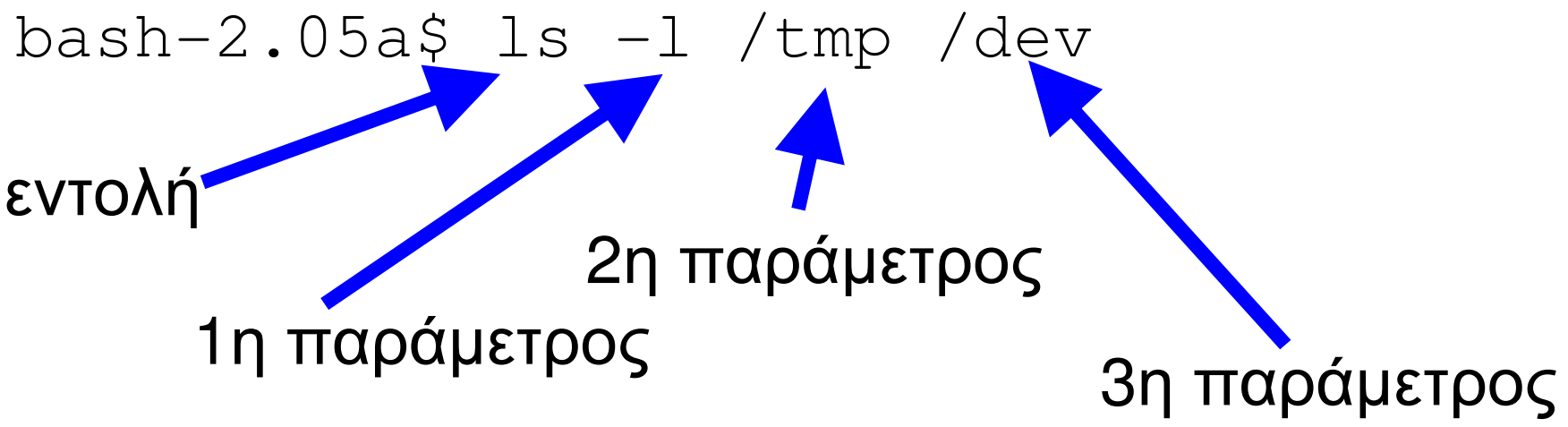
first_num=0
second_num=0

echo -n "Enter the first number --> "
read first_num
echo -n "Enter the second number -> "
read second_num

echo "first number + second number = $((first_num + second_num))"
echo "first number - second number = $((first_num - second_num))"
echo "first number * second number = $((first_num * second_num))"
echo "first number / second number = $((first_num / second_num))"
echo "first number % second number = $((first_num % second_num))"
echo "first number raised to the"
echo "power of the second number   = $((first_num ** second_num))"
```

Χρήση Command-Line Parameters

- Κάθε φορά που το κέλυφος διερμηνεύει μια εντολή συνδέει ονόματα μεταβλητών σε κάθε παράμετρο της γραμμής εντολών. Ως παράμετροι της γραμμής εντολών θεωρούνται ακολουθίες χαρακτήρων διαχωριζόμενες με κενά ή tab. (χρησιμοποιείστε εισαγωγικά αν πρόκειται χαρακτήρες με ενδιάμεσα κενά να αντιπροσωπεύουν ένα στοιχείο).



Shell programming – διαχείριση μεταβλητών

- Ειδικές μεταβλητές φλοιού

όνομα **περιγραφή**

$1 - $9 αυτές είναι οι παράμετροι

$0 το όνομα της εντολής (όπως το έδωσε ο χρήστης)

$# το πλήθος των παραμέτρων

$? Η κατάσταση εξόδου (exit status) της εντολής που εκτελέστηκε τελευταία

$$ ο αριθμός διεργασίας του φλοιού

$* ένα string που περιλαμβάνει όλα τα ορίσματα

Παραδείγματα

test_params

```
#!/bin/sh

echo "param 0=$0"
echo "param 1=$1"
echo "param 2=$2"
```

```
bash-2.05a$ ./test_params 22 33333
param 0=./test_params
param 1=22
param 2=33333
bash-2.05a$ ./test_params "2 3 4"
param 0=./test_params
param 1=2 3 4
param 2=
bash-2.05a$ ./test_params "2 3 4" "5 6 7"
param 0=./test_params
param 1=2 3 4
param 2=5 6 7
bash-2.05a$ ./test_params 2 3 4 5 6 7
param 0=./test_params
param 1=2
param 2=3
```

Παραδείγματα

test_params

```
#!/bin/sh

echo "param 0=$0"
echo "param 1=$1"
echo "param 2=$2"
```

```
bash-2.05a$ ./test_params 2 3 4 5 6 7
param 0=./test_params
param 1=2
param 2=3
bash-2.05a$ ~/test/test_params 2 3 4 5
param 0=/home/frag/test/test_params
param 1=2
param 2=3
```

```
#!/bin/bash
# FILE: testparms
echo "all are $1 $2 $3 $4 $5 $6 $7 $8
$9 $10 $11 $12"
echo "asteraki is $*"
echo "diesi is $#"
```

```
$./testparms a b c d e f g h I j k l
all are a b c d e f g h i a0 a1 a2
asteraki is a b c d e f g h I j k l
diesi is 12
```

File: fs

```
#!/bin/sh

echo "$1 status is:"
stat "$1"
touch "$1"
echo "after the touch, $1 became:"
stat "$1"
```

```
bash-2.05a$ ./fs cc
cc status is:
cc:
        inode 21316263; dev 308; links 1; size 314
        regular; mode is rw-r--r--; uid 1143 (frag); gid 1000 (it)
        projid 0        st_fstype: xfs
        change time - Sat Jan 11 20:07:07 2003 <1042308427>
        access time - Fri Oct 20 03:36:28 2006 <1161304588>
        modify time - Sat Jan 11 20:07:07 2003 <1042308427>
after the touch, cc became:
cc:
        inode 21316263; dev 308; links 1; size 314
        regular; mode is rw-r--r--; uid 1143 (frag); gid 1000 (it)
        projid 0        st_fstype: xfs
        change time - Mon Nov 27 19:36:17 2006 <1164648977>
        access time - Mon Nov 27 19:36:17 2006 <1164648977>
        modify time - Mon Nov 27 19:36:17 2006 <1164648977>
```

Εντολές προγραμματισμού στο κέλυφος

```
for i in λίστα λέξεων;
do
    εντολές, το $i παίρνει διαδοχικά τις τιμές της
  λίστας λέξεων
done
```

Παράδειγμα :

```
#!/bin/bash
for i in word1 word2 word3; do
    echo $i
done
```

Παράδειγμα

File: test2

```
#!/bin/sh

echo first test
for i in 1 2 3 4; do
        echo "i is $i"
done
echo second test
for i in "1 2 3 4"; do
        echo "i is $i"
done
echo third test
for i in "1 2" "3" 4; do
        echo "i is $i"
done
```

```
bash-2.05a$ ./test2
first test
i is 1
i is 2
i is 3
i is 4
second test
i is 1 2 3 4
third test
i is 1 2
i is 3
i is 4
bash-2.05a$
```

Παράδειγμα

File:sc1

```
#!/bin/sh
echo "params: $#"
for i in $@;do
    echo "i is $i"
done
echo “param1 is $1"
echo "param2 is $2"
echo "param3 is $3"
```

```
bash-2.05a$ ./sc1 aaaa 22222 ccc
params: 3
i is aaaa
i is 22222
i is ccc
param1 is aaaa
param2 is 22222
param3 is ccc
bash-2.05a$ ./sc1 aaaa "22222 ccc"
params: 2
i is aaaa
i is 22222
i is ccc
param1 is aaaa
param2 is 22222 ccc
param3 is
bash-2.05a$
```

Παράδειγμα

File:sc2

```
#!/bin/sh
echo "params: $#"
for i in $*;do
    echo "i is $i"
done
echo “param1 is $1"
echo "param2 is $2"
echo "param3 is $3"
```

```
bash-2.05a$ ./sc2 aaaa 22222 ccc
params: 3
i is aaaa
i is 22222
i is ccc
param1 is aaaa
param2 is 22222
param3 is ccc
bash-2.05a$ ./sc2 aaaa "22222 ccc"
params: 2
i is aaaa
i is 22222
i is ccc
param1 is aaaa
param2 is 22222 ccc
param3 is
bash-2.05a$
```

Παράδειγμα

File:sc3

```
#!/bin/sh
echo "params: $#"
for i in “$*”;do
    echo "i is $i"
done
echo “param1 is $1"
echo "param2 is $2"
echo "param3 is $3"
```

```
bash-2.05a$ ./sc3 aaaa 22222 ccc
params: 3
i is aaaa 22222 ccc
param1 is aaaa
param2 is 22222
param3 is ccc
bash-2.05a$ ./sc3 aaaa "22222 ccc"
params: 2
i is aaaa 22222 ccc
param1 is aaaa
param2 is 22222 ccc
param3 is
bash-2.05a$
```

Παράδειγμα

File:sc4

```
#!/bin/sh
echo "params: $#"
for i in “$@”;do
    echo "i is $i"
done
echo “param1 is $1"
echo "param2 is $2"
echo "param3 is $3"
```

```
bash-2.05a$ ./sc4 aaaa 22222 ccc
params: 3
i is aaaa
i is 22222
i is ccc
param1 is aaaa
param2 is 22222
param3 is ccc
bash-2.05a$ ./sc4 aaaa "22222 ccc"
params: 2
i is aaaa
i is 22222 ccc
param1 is aaaa
param2 is 22222 ccc
param3 is
bash-2.05a$
```

Παράδειγμα

File: ff

```
#!/bin/sh
for i in "$@"; do
        echo ------------------
        ls -l $i
        wc -l $i
        file $i
        echo ------------------
done
```

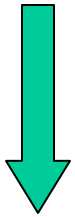
```
bash-2.05a$ ./ff file1 file2
----------------------
-rwxr--r--    1 frag it           77 Nov 20 19:17 file1
             4 file1
file1:          /bin/sh -x script text
----------------------
----------------------
-rwxr-xr-x    1 frag it           61 Nov 20 19:34 file2
             7 file2
file2:          /bin/sh script text
----------------------
```

Παράδειγμα

File: ff

```
#!/bin/sh
for i in "$@"; do
        echo ------------------
        ls -l $i
        wc -l $i
        file $i
        echo ------------------
done
```

Το file* αντικαθίσταται από το shell από τα file1 file2



```
bash-2.05a$ ./ff file*
----------------------
-rwxr--r--    1 frag it           77 Nov 20 19:17 file1
              4 file1
file1:          /bin/sh -x script text
----------------------
----------------------
-rwxr-xr-x    1 frag it           61 Nov 20 19:34 file2
              7 file2
file2:          /bin/sh script text
----------------------
```